

Scalable Deployment of a Stock Price Prediction System on Amazon Web Services (AWS)

Module Assignment for

CS5024 - Theory and Practice of Advanced AI Ecosystems

Student Name : **SAI KEERTHANA GANDIKOTA**

Student ID : **23192046**

Revision Timestamp: 02/05/2024 23:25:28

Abstract

The aim of the project is to create a scalable XGBoost model for stock price prediction, which will be deployed on the Amazon Web Services (AWS) platform. The main drive of this project comes from the important role exact share price predictions should play when making important financial decisions. AWS AI services help us to achieve scalability, reliability, and performance, which in turn make it possible for the system to deal with large amounts of data and growing user base, without any problems.

Our interaction with the AWS AI Ecosystem is carried out through various tools which includes Amazon S3 for data storage, Amazon SageMaker for model training and deployment, Amazon Lambda for serverless processing and Amazon API gateway for smooth endpoint integration. This is a strategic approach that is in line with standard industry practice as AWS is renowned for excellence in scale addressed challenges. As a result, the project is not only about solving a crucial business issue but also that it helps our portfolio by demonstrating competency in deploying latest technologies to solve real-world issues.

Contents

Abstract 2

Introduction 3

AI Ecosystem Architecture Used 4

Model Description 5

Scalability Considerations 9

References 10

Figure 1: Architecture used to create AI/ML ecosystem for Model Deployment of a Stock Price Prediction System on AWS 4

Figure 2: Amazon S3 Bucket name 5

Figure 3:Lambda Function Handler Creation 7

Figure 4: Subscription Triggered 7

Figure 5: Email 7

Figure 6:Subscription confirmation 7

Figure 7: Lambda Function 8

Figure 8: Email with Prediction Result 8

Figure 9:Function Overview 8

Figure 10: After API Gateway permission to invoke the Lambda function 8

Introduction

Suppose you are an investor whose strategies are built upon the technicalities of the stockmarkets, and hence, you are in search of faithful lessons to drive your judgments. This is what you're confronted

day in and day out on screens, tons of historical stock data, the dynamic market movements that bear no resemblance to one another, and the forecast of where prices will be headed. This is where the AI/ML problem comes into play: could you explain what methods of machine learning you plan to use for constructing a scalable system for predictions of stock prices that not only is based on historical information but also is capable of keeping pace with the dynamic market processes in real time?

I present you the Amazon Web Services (AWS) AI Ecosystem, a distinguishable array of services specially developed for meeting exactly these kinds of complexities. The first step is taking advantage of services offered by AWS like Amazon S3 for secure and scalable data stores, Amazon SageMaker for building, training and deploying machine learning models at scale as well as Amazon Lambda for executing code without provisioning and maintaining servers and Amazon API Gateway for creating, deploying and caring for APIs at scale.

In a functional way, the main objective is to help investors use a platform that is user centered and provides them with the ability to tap quality stock predictions without much hassle. We design a system that fully matches the organization's work flow, provides timely information, and, when needed, flexibly adapts to the dynamic market. In Non-functionally, methodology, scalability, reliability and efficiency are in the heart of it. We give preference to the design of a robust architecture, which can support big data processes as well as service availability and fault tolerance, and further to computational resources management.

So, the thinking behind the key features and the desired solution shall holistically supply a solution that will have taken into consideration the immediate needs of the investors as well as the the future ability to scale up and innovate.

AI Ecosystem Architecture Used

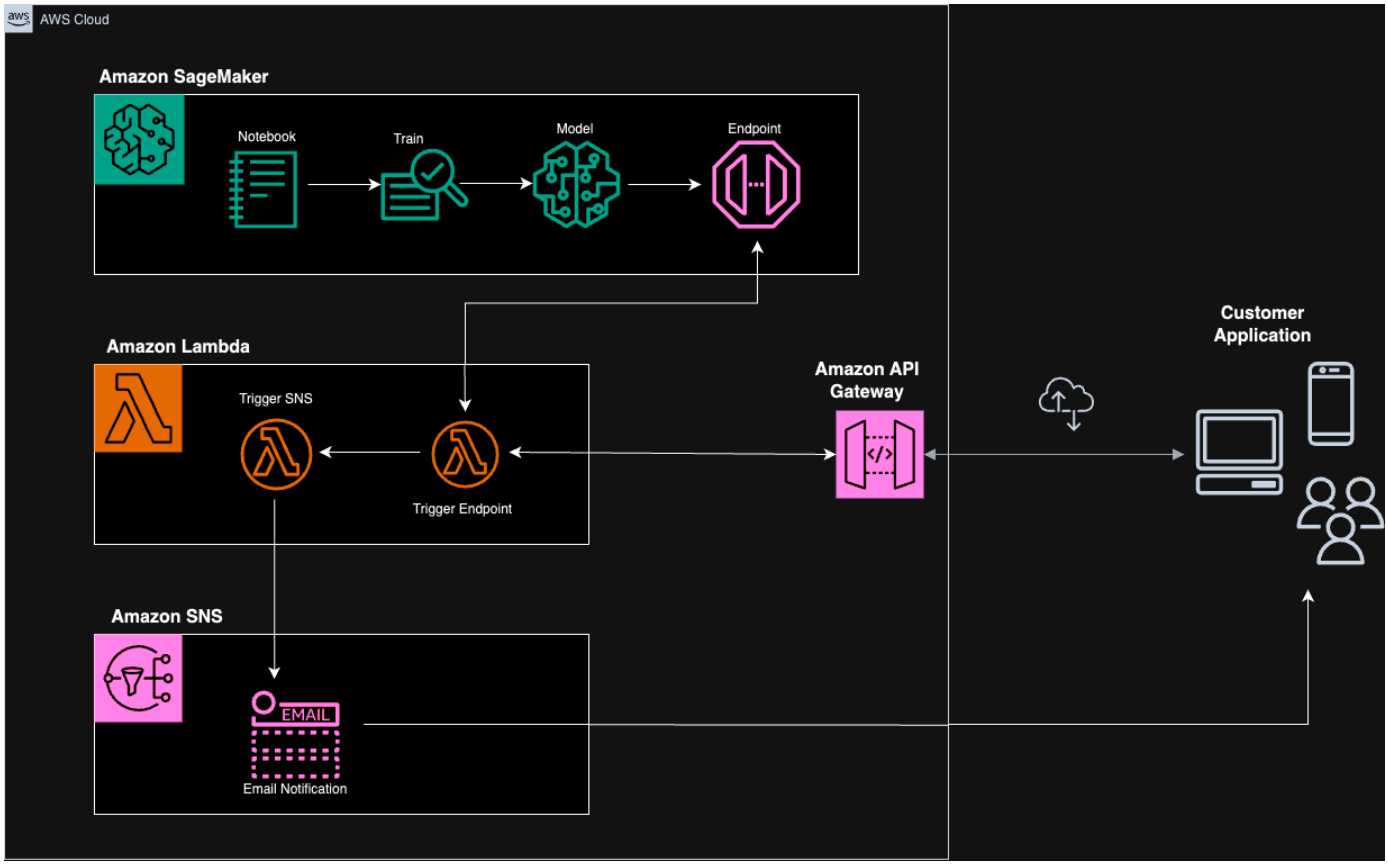


Figure 1: Architecture used to create AI/ML ecosystem for Model Deployment of a Stock Price Prediction System on AWS

The architecture includes the following components:

1. **Client Application Interaction**:

End-users interact with a client application, sending REST-style requests to an API Gateway endpoint.

2. **API Gateway and Lambda Function**:

API Gateway triggers a Lambda function upon receiving a request from the client application.
The Lambda function is responsible for:

- Mapping the request to a format required by the Amazon SageMaker endpoint.
- Invoking the SageMaker endpoint to obtain an inference from the machine learning model.
- Mapping the inference response received from the SageMaker endpoint.
- Sending the mapped response back to the client via API Gateway.
- Sending an email notification using Amazon SNS.

3. **Amazon SageMaker Endpoint**:

Hosts the trained machine learning model and provides a RESTful API endpoint for real-time inference.
Receives requests from the Lambda function and returns predictions based on the input data.

4. **Amazon SNS (Simple Notification Service)**:

Configured to send email notifications to end-users.
Triggered by the Lambda function to send notifications containing relevant information based on the prediction results.

In this solution, an IAM (Identity and Access Management) role is utilized to grant necessary permissions to each component. The IAM role defines the permissions and policies required for the Lambda function to interact with other AWS services, such as SageMaker and SNS. This ensures that each component can perform its designated tasks securely and efficiently.

Flow of the Solution:
1. The client interface is the point of contact of users with the system, where they place their orders for the prediction of stock price.
2. The client application does a REST-style interaction with the API Gateway endpoint.
3. API Gateway invokes a lambda function as a reaction to obtaining the request.
4. The Lambda function carries out the request operation, queries the SageMaker endpoint for inference, and obtains the prediction results.
5. The Lambda function will transform the prediction response and jump send it back to the client via API Gateway.
6. In parallel, Lambda function generates an email sent to the end-user with the prediction results through acquiring Amazon SNS.
7. The client now has in hand the prediction response as sent from the API Gateway and the email notification from Amazon SNS.

Here, we show how the data flow of AI-powered AWS element effortlessly ingrates different AWS service to perform real-time stock predictions and email users notifications.

Model Description

The implemented model looks into the prediction of stock prices with machine learning methods such as XGBoost model which is executed in Amazon Web Services (AWS). The model makes use of AAPL historical stock price information for a period of one year between January 1, 2019, to January 1, 2021, retrieved from the Yahoo Finance API.

First, an AI/ML code was designed which simulated a machine learning model using XGBoost to forecast stock trend according to historical data. The process includes obtaining data, cleaning up, storing the features, model training and deployment, inference and conclusion. (eMaster Class Academy, 2022)

1. S3 Bucket Creation:
- A new S3 bucket, 'yahoofinancestockpricetesting', is created for storing the data used in training and in testing.

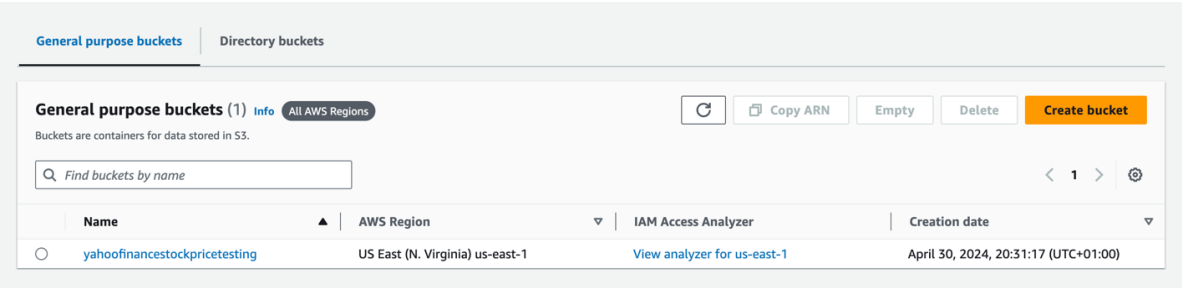


Figure 2: Amazon S3 Bucket name

2. Data Collection and Preprocessing:
- The recent historical stock price data for Apple Inc. (AAPL) is consumed throught the Yahoo Finance API using the `yfinance` library.
- Consequently, the irrelevant columns such as "Adj Close" and "Date" were ejected. (Shah, 2020)

3. Feature Engineering:
- Here the data is divided into characteristics and results. Factors are the stock prices at a given point in time, while objectives are the gains or losses in the subsequent period.

4. Data Splitting:
- Dataset is randomly shuffled and then taken into two sections called training dataset and testing dataset, where 80% of data is used for training purposes and the remaining 20% of data is used for testing purposes.

5. Data Upload to S3:
- Training and test data files placed in the S3 bucket are in CSV format.

6. XGBoost Model Building:
- A SageMaker model XGBoost is then brought up with hyperparameters `max_depth`, `eta`, `gamma` and `num_round` on.

7. Model Training:
- The model is trained on trial data, and validation on test data. The data structure is more complex compared to the basic model.

8. Model Deployment:
- Those trained model is deployed as an endpoint by SageMaker(the endpoint) using it for real-time inference.

9. Inference:
- Inference operations act on the data with the deployed model anf on the seriualized input. At the end you will get a feedback in a real time mode.

10. Lambda Integration:
- The Lambda function is going to be generated to cope with inference requests, so that the model can be run on the serverless environment.

11. API Integration:
- A parameter is planned for the input purpose which will be in this way of receiving data and returning predictions by the means of Lambda function.

12. Cleanup:
- On completion of the task, clean-up ensures that resources, namely, SageMaker endpoint and S3 bucket, which may cause additional costs, are deleted.

Deployment to the Cloud:

The model is put in the cloud using Amazon SageMaker as the deployment process. The dataset is prepared and randomly split into training and testing sets after that, and the SageMaker offers the pre-built XGBoost container for model training. The values of hyperparameters are set to optimize the model's performance, which is then deployed as the endpoint which makes real-time predictions in Amazaon SageMaker. (eMaster Class Academy, 2022)

The flow:

1. Serialized the input, then invoke the SageMaker Endpoint, and finally return the response.

Lambda function handler

```
In [27]: import boto3

ENDPOINT_NAME = 'sagemaker-xgboost-2024-05-02-11-23-56-316'
runtime = boto3.client('runtime.sagemaker')

def lambda_handler(event, context):
    inputs = event['data']
    result = []
    for input in inputs:
        serialized_input = ','.join(map (str, input))
        response = runtime.invoke_endpoint (EndpointName=ENDPOINT_NAME,
                                            ContentType='text/csv',
                                            Body = serialized_input)
        result.append(response['Body'].read().decode())
    return result

In [28]: Input_json = {'data':
            [[1.33520004e+02, 1.33610001e+02, 1.26760002e+02, 1.29410004e+02, 1.43301900e+08],
             [1.33520004e+02, 1.33610001e+02, 1.26760002e+02, 1.29410004e+02, 1.43301900e+08],
             [1.33520004e+02, 1.33610001e+02, 1.26760002e+02, 1.29410004e+02, 1.43301900e+08]
            ]
}

result = lambda_handler(Input_json, __)
result

Out[28]: ['132.52377319335938\n', '132.52377319335938\n', '132.52377319335938\n']
```

Figure 3:Lambda Function Handler Creation

2. When the SNS topic is triggered, a notification email is sent to the client. Upon confirming the subscription via email, they receive a success confirmation.

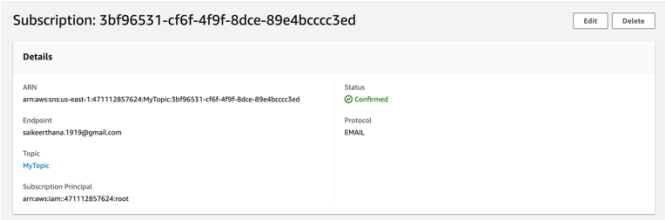


Figure 4: Subscription Triggered

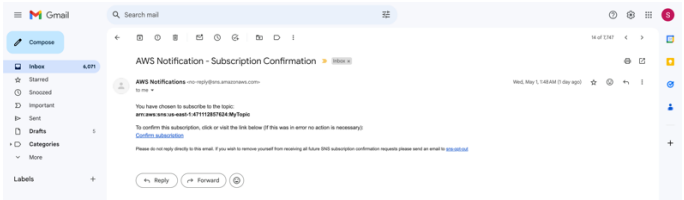


Figure 5: Email

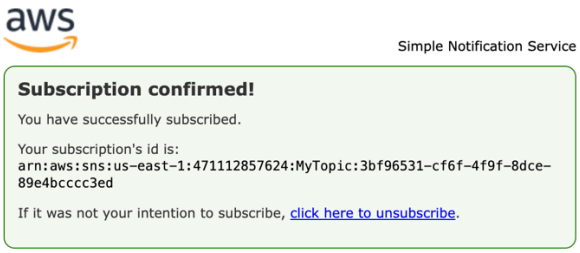


Figure 6:Subscription confirmation

3. Publish an example content and deploy the model to send an email with the prediction result to the client.

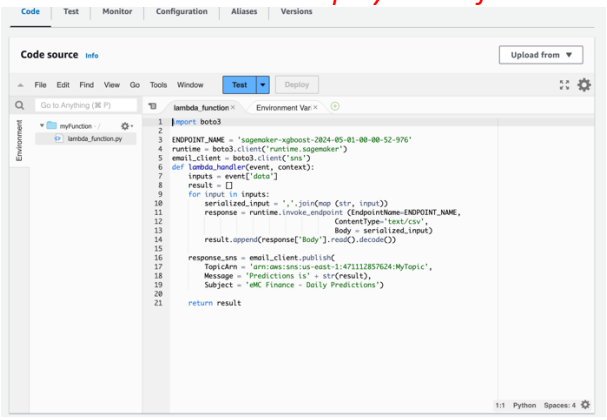


Figure 7: Lambda Function

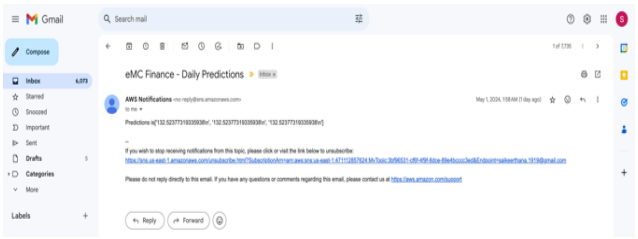


Figure 8: Email with Prediction Result

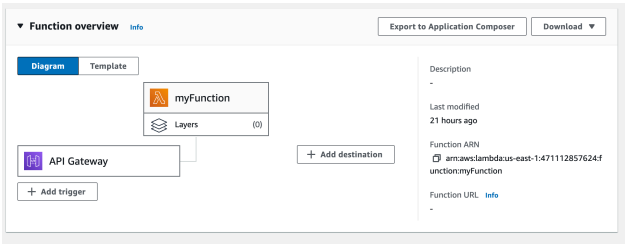


Figure 9:Function Overview

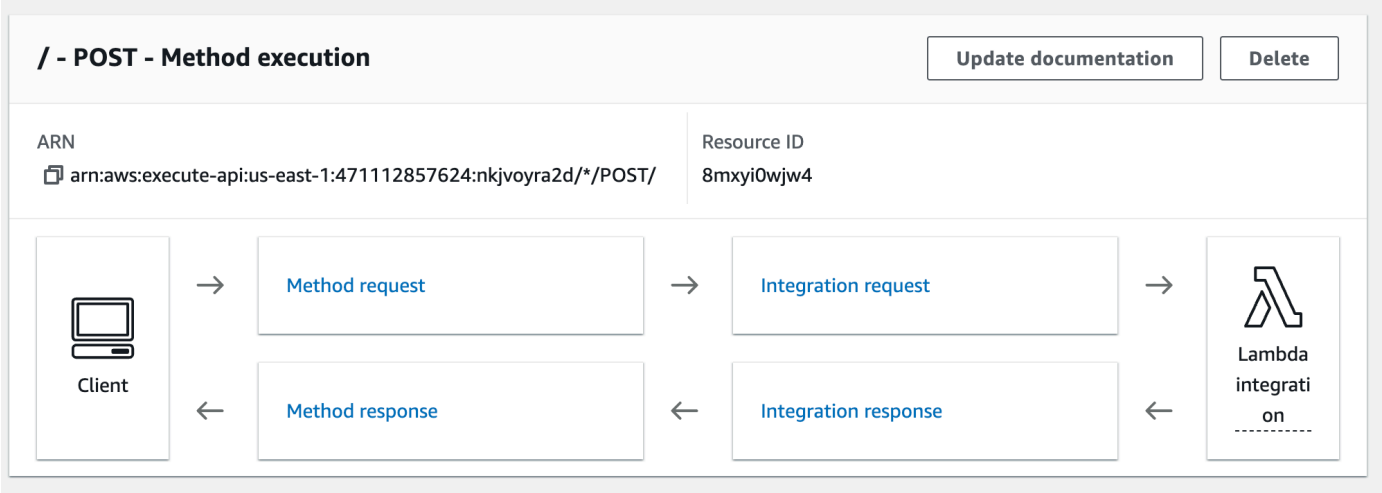


Figure 10: After API Gateway permission to invoke the Lambda function

API Gateway is configured with permission to trigger the Lambda function; therefore, communication between the services will be between the services uninterruptedly. The process starts by a client transmitting a request to the API Gateway endpoint, indicating a POST method in the HTTP_Method request parameter. The API Gateway comes next and combines the request and then proceeds by directing it to the Lambda function for processing. Post the generation of a relevant reply by Lambda, a Gateway API will integrate it with this response and return it to the client. This activity provides them with a possibility to satisfy clients inquiries and answers directly within the AWS ecosystem.

Scalability Considerations

Scalability is mainly the most important factor to note when designing a XGBoost model for the purpose of predicting stock prices. We have scalability as our system needs to be adjustable to handle a large number of transactions and account for the inclusion of shares or users in time, usability having become a critical part. Maintain of the correctness of the model as it develops and data grows is achieved by regular high performance. Resource utilization is usually optimized thereby, operations are scaled up and down depending on the customer demand, which not only boosts performance but also reduces the cost. Sharing these principles resembles to traditional pay-as-you-go model of cloud computing, which we can apply to granularly control resources and make provisions for uncertainty of demand and users' changes in behavior. Scalability, at its core, is the foundation of our application to get flexible together with emerging user and business needs, provide stability, performance, and competitiveness in the world.

When developing and use our AI model for stock price predictions, scaling needs also to be considered. This suggests a certain algorithmic accuracy of our system with respect to processing, people participating, and resources utilization.
S3 (Simple Storage Service) bucket was installed first at AWS - Amazon Web Services. The most exciting feature here is that the size of the pocket can grow with us, and we can use it to store all the data we use to train on our model.
In addition, we configure its capacity of processing large amount of data for the training phase. We select SageMaker to reduce our cost of ML algorithms operation. The ' spot instances ' that AWS offers can also be saved so that we can pay less.
After we have the model trained, we put it as SageMaker's end-point. As a serverless model, it can take many users' requests on the same time by scaling up and down the computing power, which is a behind the scene for the user.
Elsewhere in our architecture, we employ the Lambda function to process the predictions. Lambda deploys with auto-scaling so it can scale up if more people need a prediction at the same time.
We pick up the remaining components that can be recycled to save money and act friendly towards the environment. Through this thinking of scalability our system can keep pace with the development that adds more people to it and will always stay fast with excellent reliability.

References

eMaster Class Academy. (2022, February 10). AWS Machine Learning Specialization [Video playlist]. YouTube. https://www.youtube.com/playlist?list=PLMWIyphKbqfwW4RmL1G29Q_7LOnUV-cE6

Shah, I. (2020, February 7). *Historical Stock Price Data in Python. Towards Data Science.* Retrieved from https://towardsdatascience.com/historical-stock-price-data-in-python-a0b6dc826836

Amazon Web Service, AWS. (n.d.). *AWS Documentation.* Retrieved from https://aws.amazon.com/documentation/